PAPER READING

# MAI-Thinking-1

## Building a Hill-Climbing Machine

Microsoft AI technical report · June 2026 · 109 pages

Zhikai Chen

INTRODUCTION · OVERVIEW

# MAI-Thinking-1 at a glance

### MODEL

**962B total / 34.7B active** MoE, 78 layers, 256K context.

Base model MAI-Base-1.
Report and release: June 2026.

### DATA AND COMPUTE

**30T** pre-training + **3.55T** mid-training tokens on **8,192 GB200**.

RL on GB300: 4,096 inference + 768 learner GPUs at peak.

### RECIPE

From scratch on licensed or public human-written data.

No third-party distillation; RL starts with no reasoning traces.

### HEADLINE RESULTS · PASS@1, MEAN OF 4 RUNS · VS. SONNET 4.6

| | AIME 2025 | GPQA Diamond | LCB v6 | SWE-bench Verified | SWE-Bench Pro | Terminal-Bench |
|---|---|---|---|---|---|---|
| **MAI** | **97.0** | **84.2** | **87.7** | **73.5** | **52.8** | **46.0** |
| Sonnet 4.6 | 95.6 | 89.9 | – | 79.6 | – | 59.1 |

# Against MoE models of similar scale

| Model | Released | Total | Active | Act. ratio | Routed + shared | Exp. sparsity | MoE layers |
|---|---|---|---|---|---|---|---|
| **MAI-Thinking-1** | Jun 2026 | 962B | 34.7B | 3.6% | 8/512 + 0 | **64**× | 39 / 78 |
| Kimi K2 | Jul 2025 | 1T | 32B | 3.2% | 8/384 + 1 | 48× | 60 / 61 |
| DeepSeek-V3.2-Exp | Sep 2025 | 671B | 37B | 5.5% | 8/256 + 1 | 32× | 58 / 61 |
| DeepSeek-V4-Pro† | Apr 2026 | 1.6T | 49B | 3.1% | 6/384 + 1 | 64× | – / 61 |
| Llama 4 Maverick† | Apr 2025 | 400B | 17B | 4.3% | 1/128 + 1 | 128× | 24 / 48 |
| Qwen3-235B-A22B† | Apr 2025 | 235B | 22B | 9.4% | 8/128 + 0 | 16× | 94 / 94 |

**READ IT THIS WAY**

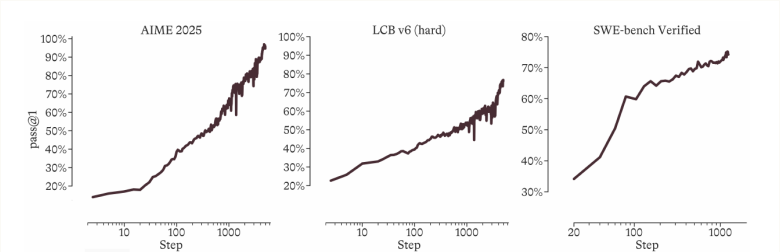
Sparsest MoE *layers* among top-8 designs (DS-V4-Pro reaches the same 64× with top-6), but only half the layers are MoE, so the overall ratio sits next to Kimi K2.

**CLOSEST RELATIVE**

Llama 4 Maverick also alternates dense and MoE, but routes top-1 with a shared expert; MAI routes top-8 with none.

Sources: MAI Table 1; Kimi K2 and DeepSeek official configs. †Public model cards.

# The hill-climbing machine




Log-linear RL climbs over thousands of steps. Left, center: STEM climb. Right: agentic climb. Fig. 1 (p.1).

**LEARNED, NOT INHERITED**
MAI's bet: distilled skills are too brittle for a long climb.

**SIMPLICITY IS SUSTAINABLE**
Simple recipes, clean trusted data, transparent infra.

**RIGOR AVOIDS SHORTCUTS**
Every decision tested on ladders and held-out evals.

INTRODUCTION · THESIS

# Three claims this talk will make

1 **Decide small, check big, with the right yardstick.** Parts 2–6

Every pre-training choice is priced in compute saved, on small models that copy the big run's shape and data repeats. Some wins held at 1T; some flipped.

2 **Design for the machine, not for FLOPs.** Parts 1, 7, 10

Half the layers are dense because all-to-all traffic, not arithmetic, sets the step time. Bit-exact restarts and planned weight sync keep thousands of GPUs checkable and busy.

3 **Keep RL alive for thousands of steps.** Parts 8–10

From a base that never saw reasoning traces: a wide clip, an entropy knob, a learner that replays the sampler, re-distillation after collapses.

**Listen for** what the paper measures and what it only asserts; open questions come at the end.

PART 1 OF 10

# Model architecture

② Design for the machine

A trillion parameters at 35B of compute per token. The hard part is not the math; it is moving tokens between GPUs.

**IN THIS PART**

The design · why MoE: knowledge vs. compute · how MoE runs on GPUs · why: sending costs time · two levers · the evidence

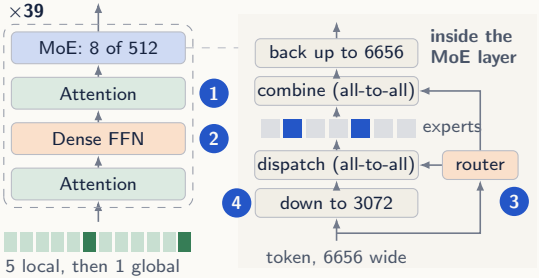

**PRE-TRAINING**

1 **Model architecture**
2 Ablations: what transfers
3 From ladder to real run
4 Pre-training data
5 Mid-training
6 Training recipe
7 YOLO training infrastructure

**POST-TRAINING**

8 RL key technologies
9 Three climbs
10 RL infrastructure: Rocket

# Alternate a dense layer with a very sparse one



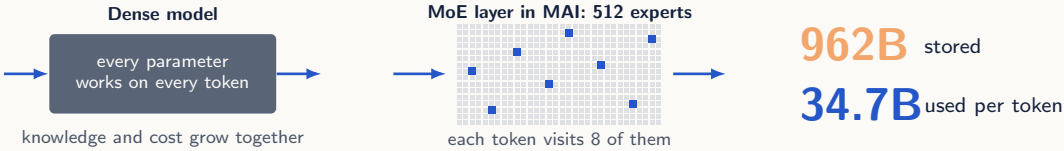

78 layers. GQA: 80 query heads share 8 key/value heads. QK-Norm: queries and keys are normalized. o200k tokenizer.

**Compare.** DeepSeek-V3 and Kimi K2 put an MoE in nearly every layer; MAI makes half its layers dense.

1. **5 local : 1 global attention** (Gemma 3). Local layers see the last 512 tokens; every sixth sees all.
2. **Dense, then sparse.** The feed-forward part alternates: a small dense FFN, then an MoE layer with 512 experts, 8 per token. 39 of 78 layers are MoE.
3. **Router sees the full vector.** It scores 512 experts from all 6656 numbers and keeps 8.
4. **Shrink, send, expand.** In an MoE layer the token is cut to 3072 numbers, sent to its experts on other GPUs, then expanded to 6656.

**Next: why MoE at all, then why 2 and 4 (both cut traffic between GPUs).**

# MoE decouples knowledge from compute




### KNOWLEDGE FOLLOWS STORED PARAMETERS

Physics of LMs 3.3 (Allen-Zhu & Li, 2024): about 2 bits of facts per parameter. A 32-expert MoE keeps capacity within $1.3\times$ of dense, using 8.8% of its parameters per token.

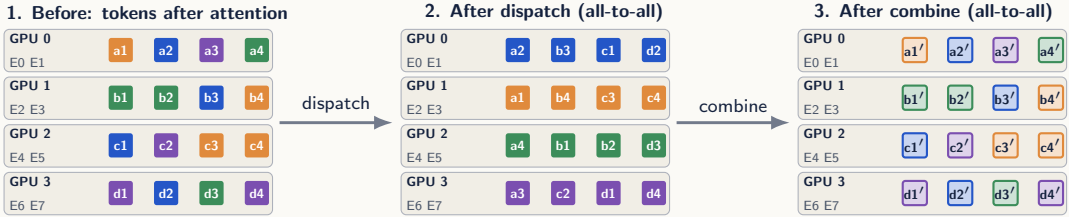
### REASONING FOLLOWS ACTIVE COMPUTE

Mixture of Parrots (Jelassi et al., ICLR 2025): more experts at a fixed active size keep improving memorization; reasoning saturates. Some graph tasks need a wider layer, not more experts.

**So: 962B parameters to store knowledge, 35B of compute per token to think with.**

# EP: move the tokens to the experts

**The setup.** The experts do not fit on one GPU, so each GPU keeps a few and the tokens travel to them. Toy: 4 GPUs, 8 experts, each token picks 1.




**Router decides.** Colour = the GPU that holds the token's expert. Every GPU starts with a **mix** of colours.

**Dispatch.** Each GPU sends every other GPU a **different** packet: GPU 1 sends b3 to GPU 0, b1 and b2 to GPU 2, keeps b4. Then experts run.

**Combine.** Results go home and are added up, weighted by the router.

**In MAI.** 512 experts on 64 GPUs (8 each), 8 per token; tokens travel as 3072 numbers. EP: GShard (2020).

# The all-to-all costs time, not FLOPs

**One layer, as time on one GPU**




**orange**: the all-to-all of the last slide. No math, so FLOPs miss it, but the next layer waits for it.

In MAI: 39 MoE layers, each with a send out and a send back, forward and again backward.

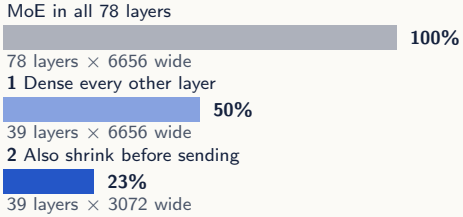
1. **Why tokens travel.** 962B do not fit on one GPU, so a layer's experts are spread over 64 GPUs and every token is shipped to its experts and back (last slide).
2. **Why it hurts.** Shipping is not math. A design can look cheap in FLOPs and still be slow in hours.

**An MoE model has two costs: math and sending. FLOPs count only the math.**

# MAI's design cuts the sending to about a quarter

**Bytes sent per token, counted by hand**
Each MoE layer sends a token to 8 experts and back, so

bytes $\propto$ (number of MoE layers) $\times$ (width sent)



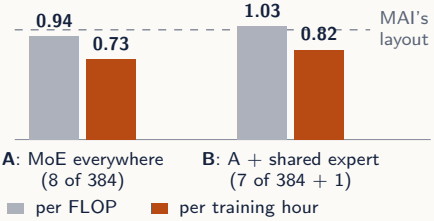

1. **A dense layer between MoE layers.** A dense FFN runs on the token's own GPU, so it sends nothing. To keep 962B parameters with half as many MoE layers, each MoE layer gets more experts: 512, of which a token uses 8.
2. **Shrink before sending** (LatentMoE, NVIDIA 2025). Project the token from 6656 down to 3072 numbers, send that, let the experts work at 3072, project back up after the return trip. The router still chooses from the full 6656.

**Also:** all-to-alls stay inside one 64-GPU NVLink rack, off the slower links between racks.

**Compare.** DeepSeek-V3 keeps MoE in nearly every layer and instead limits each token to 4 nodes.

# Per FLOP a tie; per hour, MAI's layout wins

**The test.** Two rivals put an MoE in *every* layer (same total-to-active ratio as MAI), each scored against MAI's layout = 1.0. A **shared expert** is one expert that every token always uses: an always-on dense path.




Efficiency gain, weighted average over eval categories. Below 1 = worse than MAI's layout.

**Q What does it show?**

A loses on both rulers. B ties per FLOP (1.03) but gets 18% less done per hour (0.82): twice the MoE layers, twice the sending (previous slide).

**Q So does MAI use a shared expert?**

**No.** Its dense layers already are the always-on path, and adding a shared expert gave little or no gain. Final: 8 of 512 routed experts, no shared one.

**Compare.** DeepSeek-V3 and Kimi K2 keep a shared expert in every MoE layer; Qwen3's MoE models drop it, like MAI.

PART 2 OF 10

# Ablations: what transfers

① **Decide small, check big**

Only small models are affordable to test. Which results survive the trip to 1T, and which do not?

**IN THIS PART**

The ladder · what to measure · efficiency gain · held up: experts, balancing · did not transfer

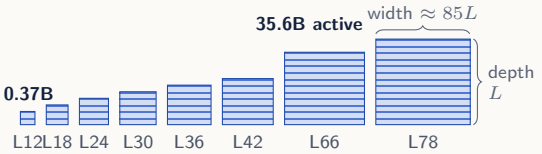

**PRE-TRAINING**

1 Model architecture
2 **Ablations: what transfers**
3 From ladder to real run
4 Pre-training data
5 Mid-training
6 Training recipe
7 YOLO training infrastructure

**POST-TRAINING**

8 RL key technologies
9 Three climbs
10 RL infrastructure: Rocket

# A ladder is one design trained at eight sizes




Dark lines: one block of 5 local + 1 global layer. MAI-Base-1 is the L78 model, resized slightly for DeepEP (34.7B active). Every size trains at the same tokens per parameter, so scale is the only thing that changes along the curve.

**In the literature.** DeepSeek-V2, Gemma 2 9B also use ≈85. Levine (2020): the best shape drifts with size; 85 is a default.

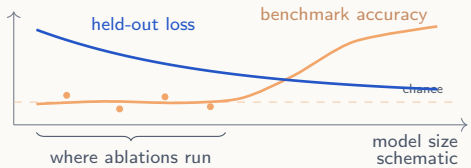
**Q What is a ladder?**
One design at eight sizes (0.37B–35.6B active); every change runs at every size.

**Q How do the sizes differ?**
Only in depth $L$; width $= 256/3 \times L \approx 85L$.

**Q Is a fixed ratio safe?**
Kaplan (2020): at a fixed size the ratio barely moves loss.

**Q Why no $\mu$P?**
$\mu$P transfers the LR across width only; the ladder also grows depth, and the main run trains several times longer, which lowers the best LR (Bjorck 2025). MAI does not say how it sets LR per size.

# Small models need a smooth signal




**How the ∼40 loss sets are combined into one score**




Multi.

**Q Why not benchmarks?**

At ladder sizes most benchmarks sit near chance, so the differences between two variants are noise.

**Q What instead?**

Loss on ∼40 held-out sets (vendor STEM and math, private code, earlier models' logs). Code is half the score, and 55% of the mix.

**Q What can go wrong?**

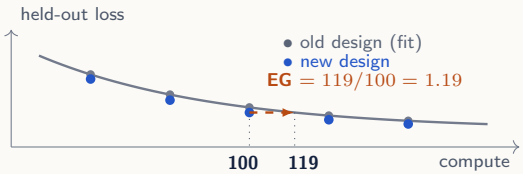
Low loss does not guarantee good behavior after RL (cases at the end of this part).

**In the literature.** Schaeffer (2023): "emergent" jumps often come from all-or-nothing metrics. Du (2024): equal loss, equal scores.

# Score every change by the compute it saves

**The question.** Is a change worth keeping? A loss gap like $2.31 \to 2.30$ is hard to read, and it shrinks as models grow. So convert it into compute.

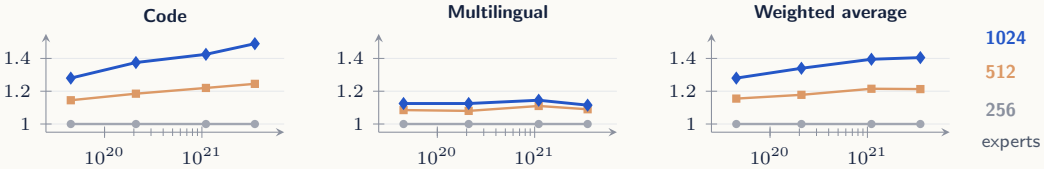
1. Train the old design at several sizes; fit loss against compute (grey line).
2. Train the new design at the same sizes (blue dots).
3. Read across: a blue dot reaches its loss with 100 units of compute; the grey line needs 119 for the same loss. EG $= 119/100 = 1.19$.
4. Keep the change only if EG $> 1$ at every size.




The new design is worth 19% more compute. Same idea for hours: replace compute by training time (Part 1's per-hour score).

**In the literature.** Kaplan (2020) and Chinchilla (2022) supply the loss-vs-compute line; Moonlight's "Muon $\approx 2\times$ compute" is the same kind of number.

# More experts keep paying off as compute grows




EG over the 256-expert ladder (= 1) vs. training FLOPs. Redrawn from Fig. 3 (p.9), three of six panels.

**The test.** Keep top-8 and the expert size; add experts: 256, 512, 1024. More stored parameters, same compute per token. If this only helped small models, the lines would sink toward 1.

**The result.** They rise with compute on code and on the weighted average. Only multilingual is flat.

**In the literature.** Abnar (2025): sparser wins at fixed compute. Kimi K2: 384 experts.

**SO WHY STOP AT 512?**

Not because the gains ran out: 1024 was still better. 512 balanced quality against all-to-all traffic and serving cost.

# Balance the global batch, not each micro-batch

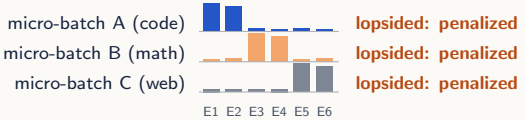
**BALANCING LOSS**
A penalty when a few experts get most of the tokens.

**MICRO-BATCH**
One GPU's 16K tokens per step: often one document.

**GLOBAL BATCH**
All GPUs together: 134M tokens, every domain.

**The question:** which tokens count toward each expert's share?

**Count per micro-batch (most frameworks, per Qiu et al. 2025)**




router pushes code tokens onto math/web experts

**Count over the global batch (MAI, Qwen)**




**A + B + C is even: no penalty**

E1–2 keep code, E3–4 math, E5–6 web

**MAI's finding:** *where* you count mattered more than *which* method.

# Allow local skew, so drop no tokens

**One step: 6 code tokens pick expert 7**

**Capacity limit of 4**




2 tokens dropped: they skip expert 7

**Dropless (MAI): overflow waits a round**

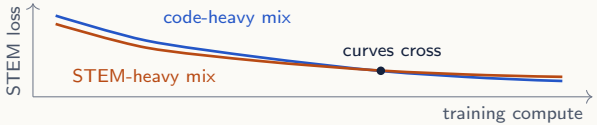



all 6 processed; memory never over 4

1. **Skew is allowed now.** Global balancing lets a code-heavy step crowd the code experts.
2. **A capacity limit drops the overflow.** Each expert takes a fixed number of tokens per step (v2: $2\times$ its fair share); the rest skip it.
3. **Dropping bends experiments.** With it, balancing ablations picked other winners.
4. **So MAI drops nothing** (nor does DeepSeek-V3). Overflow waits a round; GPUs swap token counts first.

**Result.** v2 → v3 (capacity $2\times$ → dropless, same architecture): EG $1.40\times$ over v2, other changes included; MFU held, as capacity padding vanished. MegaBlocks (2023) made dropless efficient.

# Data rankings can flip with scale




schematic of Fig. 6 (p.18)

**THE QUESTION**

If mix A beats mix B on small models, does it still win on a big one?

**WHAT THEY DID**

Two mixes: code-heavy (∼50% code) and STEM-heavy. Each trained at 5B and at 23B active, ∼20T tokens.

**WHAT HAPPENED**

STEM-heavy won at 5B and early at 23B. Midway the curves crossed and code-heavy won.

**SO WHAT**

Suspects: two duplicate-heavy, low-diversity STEM sources (11.8% vs. 0.3%). Judge a mix by its trend.

**In the literature.** RegMix (2025) assumes rankings carry over across sizes; this is a counterexample. Part 4 shows the theory that predicts it.

2 · DID NOT TRANSFER: THE RULER MATTERS

# A cheap ruler and the real ruler can disagree

**CHEAP RULER**

FLOPs, loss on small models. Fast; used on every idea.

**REAL RULER**

Training hours, the model after RL. Slow; what MAI is graded on.

| Three disagreements | Cheap ruler said | Real ruler said | MAI followed |
|---|---|---|---|
| MoE in every layer? | Per FLOP: as good | Per hour: 18% worse | real ruler |
| Final LR 0.01× peak, as usual? | Usual choice | After RL: 0.1× scored better | real ruler: 0.1× |
| Mid-train at 32K context? | Loss: 32K is enough | After post-training: unknown | caution: 64K |

**Rule of thumb.** Filter ideas with the cheap ruler; make the final call with the real one. Sardana (2024) does the same for model size: count serving cost too, and the best model gets smaller.

PART 3 OF 10

# From ladder to real run

**① Decide small, check big**

Test runs are short; the real run is long. The tests predict it only if they re-read each data source as often as the real run does.

**IN THIS PART**

Short tests, long run · copy the repeats · what a re-read is worth

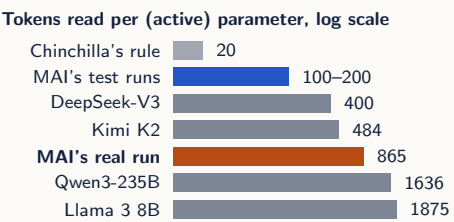

**PRE-TRAINING**

1 Model architecture
2 Ablations: what transfers
**3 From ladder to real run**
4 Pre-training data
5 Mid-training
6 Training recipe
7 YOLO training infrastructure

**POST-TRAINING**

8 RL key technologies
9 Three climbs
10 RL infrastructure: Rocket

# Test runs are short; the real run is long




Tokens and active parameters from each report.

**The catch.** A long run re-reads small sources; a short test barely reads them once (next slide).

**Q What is "tokens per parameter"?**

Text read per parameter. MAI: 30T / 34.7B active ≈ 865.

**Q Why are the tests short?**

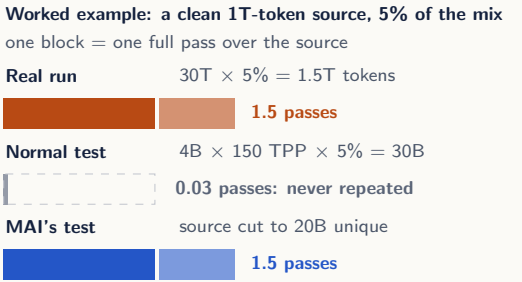
They compare ideas; 100–200 gives about the most model per unit of compute, so many ideas fit.

**Q Why is the real run long?**

A smaller model that read more is cheaper every time it runs (serving, RL rollouts), and it keeps improving: Llama 3 8B did up to 15T.

3 · COPY THE REPEATS

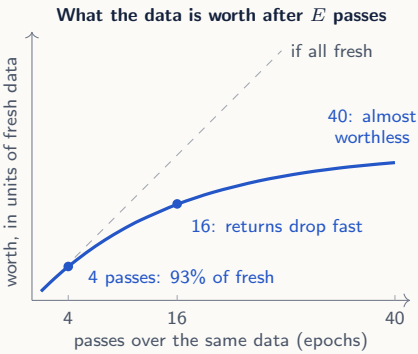
# Tests must copy the real run's re-reads




1. **Reading the same text again teaches less** than reading new text (next slide).
2. **In the real run, small clean sources are read several times**: math 5.3 times on average.
3. **A normal test reads 3% of such a source**, so it never repeats it. The source looks better in the test than it will be in the real run.
4. **MAI's fix:** in the test, cut the source down so it is read 1.5 times, just like in the real run.

**In the literature.** Goyal et al. (2024, CMU) find the same: clean data wears out, so the best data choice depends on how long you train.

# Re-reading data: fine up to about 4 times




Their fit: the $R$-th repeat is worth $e^{-R/15}$ of a fresh token.

**Q The question**
When new text runs out, does reading the same text again still help?

**Q The experiment**
About 400 models up to 9B parameters, each trained with the same text read 1 to many times.

**Q The answer**
Up to 4 reads: almost as good as new text. Then each read helps less; by ∼40 reads, not at all.

**Q For MAI**
No source is read more than 8 times, and math, the most re-read, averages 5.3.

PART 4 OF 10

# Pre-training data

① **Decide small, check big**

Data sets the ceiling. How do you weight 29T unique human-written tokens, and what does theory say about the answer?

**IN THIS PART**

Sources · the 30T mix · repetition control · the search · why the best mix changes with size · what MAI could have done

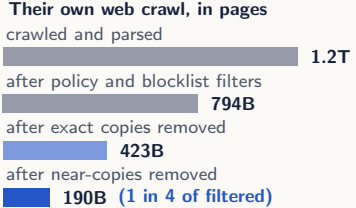

**PRE-TRAINING**

1 Model architecture
2 Ablations: what transfers
3 From ladder to real run
4 **Pre-training data**
5 Mid-training
6 Training recipe
7 YOLO training infrastructure

**POST-TRAINING**

8 RL key technologies
9 Three climbs
10 RL infrastructure: Rocket

# Human-written text only, with copies cut hard




Then a quality model drops the bottom 70% of English pages. Common Crawl: same pipeline, 300B → 24.2B pages.

**All sources: 29.2T unique tokens**

**Only human-written text.** Own crawl (it obeys robots.txt), web PDFs, licensed books and journals, public GitHub. No synthetic text, no open datasets, AI-heavy domains dropped. LLM cleaners may delete text, never write it.

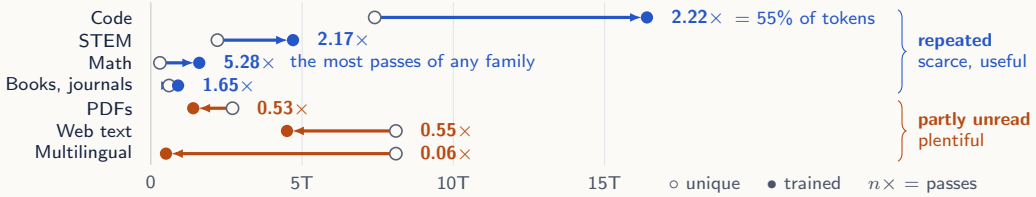
**Cut copies at every level.** Exact copies (hash), near-copies (MinHash fingerprints, on which pages 80% alike collide), template pages, same-meaning code (embedding clusters). Big sparse models memorize repeats; dedup improved both loss and reasoning scores.

**Keep benchmarks out.** Drop huggingface.co and mirrors, fuzzy 20-gram matches to public test sets, and every SWE-bench Verified PR. Leaks still slip through, hence private evals (Part 2).

**Compare.** FineWeb and DCLM (2024) also filter Common Crawl with classifiers; Phi-4 and Nemotron-CC add LLM-written synthetic text, which MAI rules out. Lee (2022): dedup cuts memorized output $10\times$.

# Weight data by value, not by supply

The question: dedup leaves 29.2T unique tokens for a 30T run. Read once each, supply would set the mix, and web plus multilingual text alone would fill over half the run.




**Each family gets the share that best lowers the code-weighted loss (how: next slide). No dataset is read more than 8 times: each repeat adds less and invites memorization.**

**Compare.** Llama 3 used about 17% code, a third of MAI's share; Muennighoff (2023): repeats lose value.

# Repetition is controlled in three places

### 1. BEFORE: DEDUP

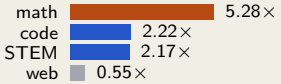
Remove exact and near copies (MinHash), boilerplate and benchmark overlap.

**Why:** repetition drives memorization, worse in big models.

**Result:** 29.2T unique tokens.

### 2. IN THE MIX: RE-READS

Scarce, useful data is re-read; plentiful data is only sampled.




Cap: 8 reads per dataset; ladder tests copy the re-reads (Part 3).

### 3. MID-TRAINING: MEMORIZED?

Same corpus, re-weighted, read again.

**Per source:** share of recent loss gain from tokens already near-certain (loss $< 0.01$).

High share = memorized: fewer re-reads.

**Plus:** dropout 0.15, weight decay 0.1 (Part 6). Up to ∼4 reads ≈ new data (Muennighoff); math's 5.3 is near the edge.

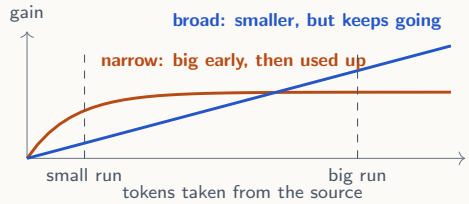
# Many small tests, then a few bigger checks

0. **One score per mix:** code-weighted loss on ∼40 held-out sets (Part 2); lower is better.
1. **Learn each source:** a small model with it at 50% of the data shows which scores it lowers.
2. **Try thousands of mixes** on 0.76–4B models, each a small change from the current mix.
3. **Alternate:** re-weight ∼10 groups (code, STEM, PDFs, web . . . ), or re-weight inside one (code files vs. commits vs. PRs). No source read over 8 times.
4. **Check bigger:** retrain a few finalists with $2.8\times$ the compute; keep the one that still wins.




**Compare.** RegMix (2025) also learns from many tiny runs; MAI adds the bigger check after a winner flipped at scale (Part 2).

# Why the best mix changes with model size




**Narrow**: small, repetitive sets; more tokens = more repeats. **Broad**: large, diverse web and code.

**The rule: give each next token to the source that still helps most.**

| Budget | Best split | Narrow share |
|---|---|---|
| small run | narrow still steep: take a lot | **high** |
| big run | narrow flat: extra goes to broad | **low** |

So a mix tuned on small models over-weights narrow data. AutoScale (2025) sees it: Wikipedia and papers win small, then stall; CommonCrawl keeps paying.

**MAI's flip (Part 2) is this case.** The STEM-heavy mix leaned on two narrow, duplicate-heavy STEM sources (11.8% vs. 0.3%). It won at 5B and lost at 23B.

# What MAI could have done: follow the trend

### WHAT MAI DID

1. Searched mixes on 0.76–4B active models and picked the best.
2. Re-checked a few finalists at $2.8\times$ that compute.
3. The risk: the flip in Part 2 appeared between 5B and 23B, beyond this check.

### WHAT THE LAST SLIDE SUGGESTS

1. Find the best mix at two sizes, see which way it moves, and extend that trend to the real size.
2. On small runs, measure how fast each source flattens. Fast-flattening sources (narrow, duplicated) get flagged before the big run.

**Is MAI exposed?** Partly. Since the flip (Part 2), data tests copy the real run's re-reads (Part 3) and finalists are re-checked bigger. Not covered: what changes between 4B tests and the 35B run.

PART 5 OF 10

# Mid-training

① **Decide small, check big**

Before RL: tilt the base toward STEM and code, and stretch it to 256K, cheaply.

**IN THIS PART**

What mid-training is · the data tilt · why it matters for RL · long context


**PRE-TRAINING**

1 Model architecture
2 Ablations: what transfers
3 From ladder to real run
4 Pre-training data
**5 Mid-training**
6 Training recipe
7 YOLO training infrastructure

**POST-TRAINING**

8 RL key technologies
9 Three climbs
10 RL infrastructure: Rocket

# Mid-training: the bridge from pre-training to RL

**Pre-training**
30T tokens, 16K context
broad mix of all sources
LR $2 \times 10^{-4} \rightarrow 2 \times 10^{-5}$

→

**Mid-training**
3.55T more tokens (≈12%)
same data, tilted to STEM and code
64K, then 256K context
LR $2 \times 10^{-5} \rightarrow 10^{-5}$, then $10^{-6}$

→

**RL climbs (Parts 8–9)**
STEM, agentic, alignment
learn from rewards, not next-token loss

**WHAT IT IS**

Still next-token prediction on documents, on a smaller, cleaner, re-weighted mix, at low learning rate and longer context. It sets what the base is best at when RL starts.

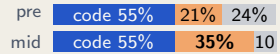
**WHAT IT IS NOT**

Not instruction tuning: no chat format, no answers to imitate. At MAI not new data either: only the pre-training corpus, filtered and re-weighted.

**Compare.** Llama 3 anneals on upsampled high-quality data; OLMo 2 mid-trains on Dolmino (incl. synthetic math); Qwen3 adds reasoning and long-context stages. Next: what goes in, then why it matters for RL.

# Mid-training tilts the mix; it adds no new data

The question: how to tilt the base toward math, code and STEM for RL, with no new data?




**1 · Re-weight.** STEM + math (orange) rise to 35%, code holds 55%, the rest falls to 10%. Short test anneals per source tune each group.



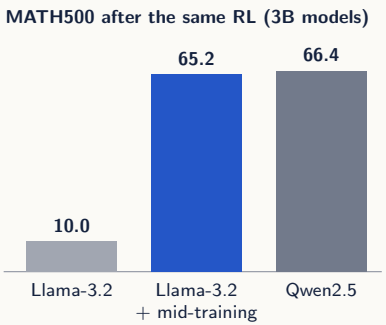

**2 · Keep reasoning-heavy text.** STEM PDFs stay only if correct and at Bloom level "analyze" or above (Bloom ranks thinking skills). Code drops web assets from low-tier repos and adds single-file views, which lowered file-level loss.




**3 · Cap what is memorized.** Sources whose recent loss drop comes mostly from near-certain tokens (loss $< 0.01$, red) get fewer passes.

**Compare.** Qwen3 adds synthetic reasoning data mid-way; MAI adds none. MAI's per-source weights come from small test anneals, like OLMo 2's microanneals.

# Mid-training sets how far RL can climb



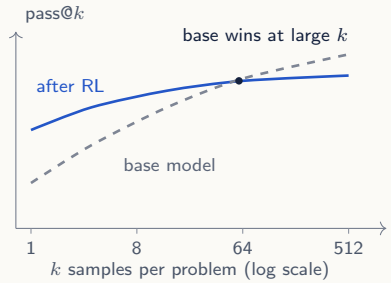

**Q The question**
The same RL recipe lifts Qwen a lot and Llama barely. What makes a base ready for RL?

**Q The experiment**
Mid-train Llama-3.2-3B: 200B tokens of good math text, then 20B reasoning-style tokens as the LR decays; same RL.

**Q The finding**
The mid-trained Llama climbs like Qwen. Data quality decides it, and long chain-of-thought data made RL verbose and unstable.

**Q For MAI**
MAI tilts its mix to STEM and code before RL, with no long-CoT or synthetic data.

# RL sharpens what the base can already sample




Schematic of Yue et al. (Tsinghua, 2025).

1. **Yue et al.:** RL wins at pass@1, but with many samples the base model solves more. RL's correct paths were already in the base's samples: RL reweights more than it adds.
2. **ProRL (NVIDIA, 2025):** long RL with KL control and reference resets pushes pass@128 past the base on many tasks, most where the base struggled; math, close to pre-training data, gained least. MAI's long climbs bet on this.
3. **CMU (Zhang, Neubig, Yue, 2025):** RL builds new skill only at the edge of what the model can already do.

**The base decides what RL can reach; mid-training moves STEM and code into reach.**

# Train at 16K, stretch to 256K only at the end

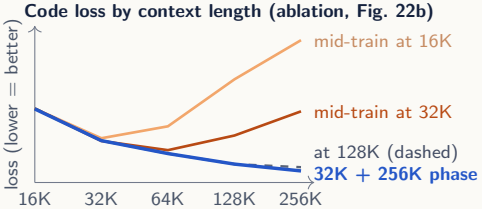
**Pre-training**
context **16K** · 30T tokens

→

**Mid-training 1**
context **64K** · 3.4T tokens

→

**Mid-training 2**
context **256K** · 150B tokens




1. **Why stage it?** Long sequences use GPUs poorly (low MFU: share of peak compute actually used), so 30T tokens at 256K would be slow.
2. **Does it cost quality?** Mid-trained at 16K or 32K, a model breaks past about $2\times$ that length. 32K plus $\sim$100B tokens at 256K matches 1T tokens at 128K; at 16–32K all runs tie.
3. **Why so cheap?** Most of the 256K gain comes in the first 1–10% of steps (Fig. 22c): a recalibration, not a new skill. Same mix, re-packed.

**Compare.** Llama 3: six stages from 8K to 128K, about 800B tokens. DeepSeek-V3: two short YaRN stages.

PART 6 OF 10

# Training recipe

① **Decide small, check big**

One 30T run, no do-overs: an optimizer, init and precision that are stable and good after RL.

**IN THIS PART**

AdamW recipe · dropout and weight decay · zero-init attention · Adam vs. Muon geometry · why not Muon


**PRE-TRAINING**

1 Model architecture
2 Ablations: what transfers
3 From ladder to real run
4 Pre-training data
5 Mid-training
6 **Training recipe**
7 YOLO training infrastructure

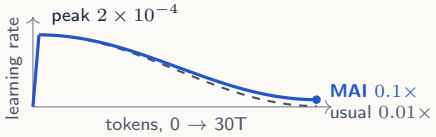
**POST-TRAINING**

8 RL key technologies
9 Three climbs
10 RL infrastructure: Rocket

# Plain AdamW, with the LR floor set after RL

> **The base model is not the product. It is judged by what RL makes of it.**

So MAI kept the standard optimizer and tuned knobs checked on the ladder or after RL.




**Betas $\beta_1/\beta_2$** usual: 0.9/0.95 → **MAI: 0.95/0.925**

$\beta_1$: memory of the gradient average (∼20 steps); $\beta_2$: of its size estimate (∼13). The RL learner even uses $0.95/0.95$.

**LR floor** usual: $0.01\times$ peak → **MAI: $0.1\times$ peak**

The cosine decay stops at $2 \times 10^{-5}$. Decaying less gave a better model *after RL*; that result decided it.

**Dropout** usual: none → **MAI: 0.15**

On each block's output, before the residual add. Unusual: PaLM and Llama pre-train without it. It helped ladder evals; MAI reuses 0.15 in self-distillation to keep entropy up.

**In the literature.** Llama 3 decays to $0.01\times$ peak. Orvieto & Gower (2025): $\beta_1 \approx \beta_2$ is robust. Xue (2023): dropout helps when data repeats (MAI's math: 5.3×).

# Dropout and weight decay: guarding re-read data

**WHAT MAI SET, AND WHY**

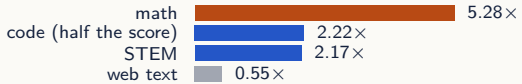
**Pre-training:** weight decay 0.1; dropout 0.15 on each block's output, because it added to weight decay and improved ladder evals.

**Self-distillation:** dropout 0.15 again, to keep entropy up and prevent collapse before RL.

**RL:** no weight decay.

Most LLMs use no dropout: each token is seen once.

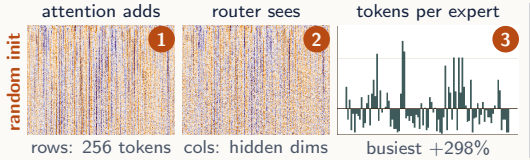
**Average reads in the 30T run**




1. **Xue (NeurIPS 2023):** re-reads hurt big models; only dropout clearly helped.
2. **Kim (ICLR 2026):** with fixed data, best weight decay $\approx 30\times$ usual.
3. **Lovelace (2026):** weight decay 1.0 cut the re-read penalty $\sim$70%.

**Is it for coding?** Not by design (it covers all data), but it guards the re-read priority data most. MAI chose dropout, Xue's lever, and kept the usual weight decay.

# Zero-init attention keeps early routing balanced




1. At step 0 the weights are random, so attention has no preference: the softmax is nearly flat.
2. Each token gets the average of the tokens before it: **causal mean pooling**.
3. Such averages look alike: all tokens get the same add-on 1, and the router sees it 2.
4. Alike tokens pick the same experts 3, some get none, and each layer makes it worse.
5. **Fix:** start the gain of attention's output RMSNorm at 0. The branch adds nothing 4; the gain still learns.
6. Each block starts as a per-token MLP; attention fades in. Load stays even 5 and EG improved.

**In the literature.** ReZero, Fixup, DiT's adaLN-Zero zero residual branches for trainability; Noci (2022): flat attention collapses tokens with depth. New here: that collapse unbalances MoE routing.

# One recipe, three rulers: SGD, Adam, Muon

$\Delta W = \arg\min_{\Delta} \langle G, \Delta \rangle + \frac{\lambda}{2}\|\Delta\|^2$: **go downhill, but stay inside a ball.** The norm picks the ball.

### SGD: EUCLIDEAN BALL



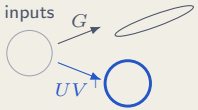

Step along $-G$: big entries move a lot, small ones barely.

### ADAM: BOX ($\ell_\infty$)




Step to a corner, $-\mathrm{sign}(G)$: every *entry* moves equally. $W$ = a bag of numbers.

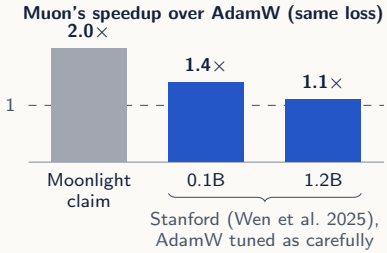
### MUON: SPECTRAL BALL




Step $-UV^\top$: every *direction* moves equally. $W$ = a linear map.

**So what is Adam?** A per-entry learning rate $1/\sqrt{v}$; with moving averages off it is $\mathrm{sign}(g)$, the max-norm steepest step (Bernstein & Newhouse 2024). **Adam signs a vector; Muon "signs" a matrix.**

# Others moved to Muon; AdamW fit MAI's stack




**Why AdamW still fit MAI** (MAI never discusses Muon)

1. **Calibration.** The ladder, EG and every knob were tuned with AdamW.
2. **Sharding.** YOLO shards optimizer state (ZeRO); Muon needs whole matrices.
3. **Yardstick.** Muon's gains are in pre-training loss, and they shrink with scale; MAI judges after RL.
4. **Continuity.** RL also runs AdamW; Moonlight saw that switching optimizers between stages hurts.

**Who switched.** Kimi K2 (added QK-Clip) · GLM-4.5 · DeepSeek-V4 (RMSNorm on q and KV, ZeRO rebuilt) · Qwen3.8-Next (router kept on AdamW).

PART 7 OF 10

# YOLO training infrastructure

② Design for the machine

Thousands of GPUs for months: fast, verifiable restarts, and speed won back after every change.

**IN THIS PART**

Bit-exact restarts and goodput · why results drift · what determinism costs · EG vs. MFU

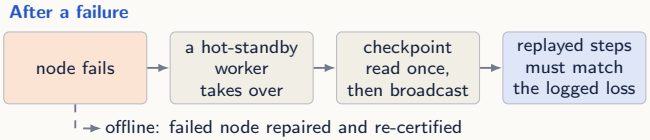

**PRE-TRAINING**

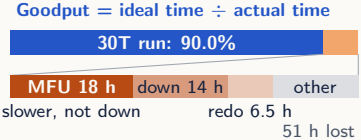
1 Model architecture
2 Ablations: what transfers
3 From ladder to real run
4 Pre-training data
5 Mid-training
6 Training recipe
**7 YOLO training infrastructure**

**POST-TRAINING**

8 RL key technologies
9 Three climbs
10 RL infrastructure: Rocket

# Bit-exact training makes every restart checkable




**THE QUESTION**

Hardware fails often on 8K GPUs. How to resume fast, and prove it is the same run?

**WHAT THEY DID**

Bit-exact training: fixed batch order, sums always in the same order, a stable top-k.

**WHAT HAPPENED**

90.0% goodput. Of 51 h lost, redoing steps took 6.5 h; MFU drops (job runs, but slower) took 18 h.

**SO WHAT**

A restart that does not match exposes a bad node or a bug. Next target: slowdowns.

**In the literature.** Goodput as in MegaScale and Llama 3 (over 90% effective time despite 419 unplanned interruptions in 54 days). Most stacks keep GPU atomics and in-switch SHARP sums for speed; MAI turns them off and pays the MFU cost.

# Why the same model can give different answers

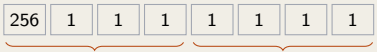
### 1. BF16 KEEPS ABOUT 3 DIGITS

$(256 + 1) + 1 = \mathbf{256}$ 257 → 256, twice

$256 + (1 + 1) = \mathbf{258}$ 258 fits

Same numbers, different order, different result.

### 2. HARDWARE DECIDES THE ORDER




one chain: **256**

two halves: **260**

exact: 263

Batch size, GPU count and kernel choice change how a sum is split, so the last bits change.

**3.** The last bits of a probability move a little. → **4.** At a near-tie, a different token wins. → **5.** Everything after it is a different answer.

**Example.** Thinking Machines (2025): Qwen3-235B, temperature 0, 1,000 runs. All agree for 102 tokens; then 992 say "Queens, New York" and 8 say "New York City".

# Bit-exact costs speed; drift costs trust

### TRAINING: MAI

Fixed reduction order, no atomics, stable top-k, NVLink SHARP off. MFU drops (no total given); deterministic attention was a bottleneck until custom kernels ($1.06\times$, then $1.14\times$).

### INFERENCE: NEURIPS 2025

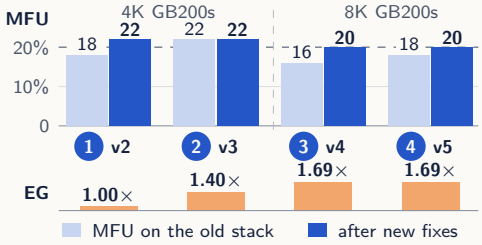
Yuan et al.: BF16 greedy decoding, R1-Distill-Qwen-7B. Accuracy moves up to 9% and answers up to 9,000 tokens with GPU count, type and batch size. FP32 compute fixes it.

### BATCH-INVARIANT KERNELS

Thinking Machines: Qwen3-235B, temperature 0, 1,000 runs gave 80 different answers; with batch-invariant kernels, one. Time: 26 s $\rightarrow$ 55 s, 42 s with a better attention kernel.

**So.** Determinism changes speed, not what the model learns ($\sim$1.6–2$\times$ slower in that test). Drift adds noise to every eval and RL ratio. MAI pays in training; in RL it replays routing instead.

# Each EG gain moved the speed bottleneck




MFU (PaLM's metric): share of peak BF16 math doing model FLOPs. EG: compute saved vs. v2 (Part 2).

1. **v2, first GB200 run.** Network (GPU-direct RDMA), attention, ZeRO-2 and kernel fixes: 18% → 22%.
2. **v3, dropless (EG 1.40×).** Token counts now move at run time; overlapping them held 22%.
3. **v4, 512 experts, top-8, LatentMoE, 8K GPUs.** Tiny expert matmuls left GPUs waiting on the CPU: 16%. FlashAttention-4 and CPU fixes: 20%.
4. **v5, 35B active, 1T total.** ZeRO-3's extra weight gathers stalled the backward pass: 18%. Activation offload restored ZeRO-2: 20%.

**Rule of thumb.** Better per FLOP can be slower per hour: budget systems work for each model change. DeepSeek-V3 likewise fits its MoE to the hardware (node-limited routing, DualPipe overlap).

PART 8 OF 10

# RL key technologies

**③ Keep RL alive**

Async RL from a base with no reasoning traces: how to keep thousands of steps from collapsing.

**IN THIS PART**

The objective · reading the clip · why a wide band · entropy · sampling · re-distill · vs. other recipes · 2026 reports


**PRE-TRAINING**

1 Model architecture
2 Ablations: what transfers
3 From ladder to real run
4 Pre-training data
5 Mid-training
6 Training recipe
7 YOLO training infrastructure

**POST-TRAINING**

8 **RL key technologies**
9 Three climbs
10 RL infrastructure: Rocket

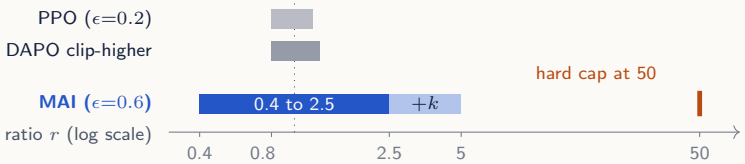
# Token-level GRPO with a wide clip and a hard cap

$$J(\theta) = \underbrace{\frac{1}{\sum_i |y_i|}}_{1} \sum_{i=1}^{G} \sum_{t=1}^{|y_i|} \min\Big( \underbrace{\hat{r}_{i,t}}_{3} \underbrace{A_i}_{2},\ \underbrace{\mathrm{clip}\left(\hat{r}_{i,t},\, 1-\epsilon,\, (1-\epsilon)^{-1}+k\right)}_{4} A_i \Big), \quad \underbrace{\hat{r}_{i,t} = \mathrm{clip}(r_{i,t}, 0, 50)}_{5}$$

1. **Every token counts the same:** divide by all tokens in the batch, as in DAPO.
2. **No critic:** $A_i = (R_i - \text{mean})/\text{std}$ over the $G$ answers to one prompt.
3. **Ratio** $r = \pi_\theta/\pi_{\text{old}}$: how much likelier the learner finds a token than the sampler did. Why it drifts: two slides on.
4. **Wide band:** $\epsilon = 0.6$ gives 0.4 to 2.5, so 40-step-old rollouts still train; $k \in [0, 2.5]$ lifts the top (next slide).
5. **Hard cap on every branch:** PPO leaves $A<0$, $r>1$ unbounded; that caused gradient spikes.
6. **Left out:** a KL penalty (as in DAPO) and an entropy bonus (worse than moving $k$).

**Compare.** Dual-clip PPO (Ye 2020) caps the same case. CISPO clips weights; GSPO uses one ratio per answer.

# Reading the clip: when a token stops learning




A token at 1% in a correct answer may rise to 1.2% (PPO), 1.28% (DAPO) or 2.5–5% (MAI) before its gradient stops.

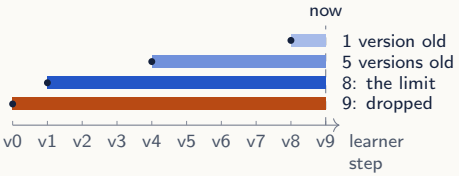
| | $r$ **rises** | $r$ **falls** |
|---|---|---|
| $A>0$ (good answer) | grows until $2.5+k$, then the gradient stops | still learns: it is being corrected |
| $A<0$ (bad answer) | still learns, with no limit ⇒ cap at 50 | shrinks until 0.4, then stops |

1. **Why 0.4 and 2.5?** $1-\epsilon$ and $1/(1-\epsilon)$: $-0.92$ and $+0.92$ in log, so up and down are treated alike. PPO's 0.8/1.2 is not.
2. **Why $k$ only on top?** The top only touches $A>0$ tokens; lifting it lets rare, rewarded tokens rise.

# Async rollouts pull the ratio away from 1

$$r_t = \frac{\pi_\theta(y_t \mid q, y_{<t})}{\pi_{\text{old}}(y_t \mid q, y_{<t})}$$

The **importance ratio**: how much more likely the learner finds token $y_t$ now than the sampler did. Weighting by $r_t$ lets slightly old data still train.




One batch, four rollouts. Dot: weights that wrote it; new weights every 5 steps.

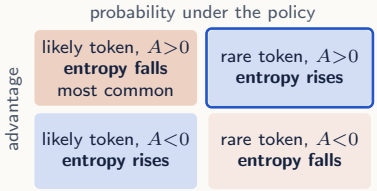
1. **Why it drifts.** Sampling and training overlap, so a rollout is a few weight versions old when used. Long ones, usually the hard problems, are the oldest.
2. **Even at zero lag,** sampler and learner kernels give slightly different probabilities (Part 10).
3. **MAI's limits.** Drop rollouts over 8 versions (40 steps) old; at long lengths, LR $10^{-6} \to 9{\times}10^{-7}$.

**In the literature.** IMPALA (Espeholt 2018) met the same lag and truncates the ratio (V-trace). AReaL caps staleness at a few steps and adds a proximal-policy anchor; MAI tolerates 40 steps with one wide clip.

# Why entropy collapses: RL rewards the likely

$$\Delta H \approx -\eta \, \mathrm{Cov}_{a\sim\pi}\left(\log \pi(a),\, A(a)\right)$$

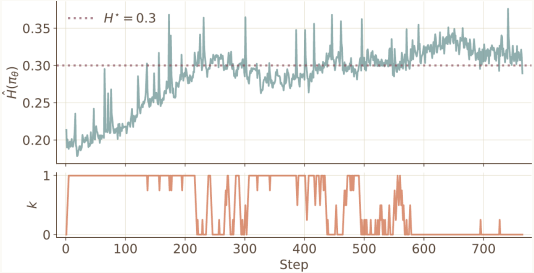
A policy-gradient step moves **entropy** $H$ (how spread out next-token choices are) by minus the covariance of log-probability and advantage (Cui et al., 2025).




1. **Why it falls.** Correct answers use tokens the model already likes, so the covariance stays positive.
2. **Why it matters.** Score is bought with entropy: $R \approx -a\, e^{H} + b$, so $H = 0$ sets the ceiling.
3. **The lever.** Raising the upper clip lets rare, rewarded tokens rise: the boxed quadrant. That is MAI's knob $k$ (next slide).

**In the literature.** Cui et al. clip only top-covariance tokens (Clip-Cov): $10^{-4}$–$10^{-3}$ of tokens reshape the curve.

# Steer entropy with the clip, not with a bonus




Top: entropy $\hat{H}$ vs. target $H^\star$=0.3. Bottom: the knob $k$ ($k_{\max}$=1 in this 800-step run). Fig. 13 (p.32).

**The usual fix: a bonus.** Add $\beta \cdot H$ to the objective, so the model is paid for spreading its next-token distribution, on every token, rewarded or not. Too small a $\beta$ and entropy collapses; too large and outputs turn random.

**MAI's fix: move the clip.** Each step, estimate $\hat{H}$ (weighted by $r_t$), then move the knob one notch:

$$k \leftarrow \mathrm{clip}\big(k + 0.25\ \mathrm{sign}(H^\star - \hat{H}),\ 0,\ 2.5\big)$$

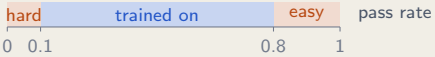
A higher ceiling $2.5{+}k$ lets good but unlikely tokens grow: entropy rises only where the reward points.

**In the run.** $k$ stays at its maximum until entropy climbs from 0.2 to 0.3 near step 200, then toggles.

**In the literature.** Skywork-OR1 (2025) runs the same $\pm$ controller on the bonus weight $\beta$ (like SAC's temperature); MAI found the bonus did worse than moving the clip.

# What gets sampled decides what the gradient sees

### PASS-RATE FILTER




If a group's answers all pass or all fail, every GRPO advantage is zero. So: 16 answers first, 128 only if 5–80% pass, train only if 10–80% pass.

DAPO filters too, but only after sampling in full.

### TOP-P MASK REPLAY

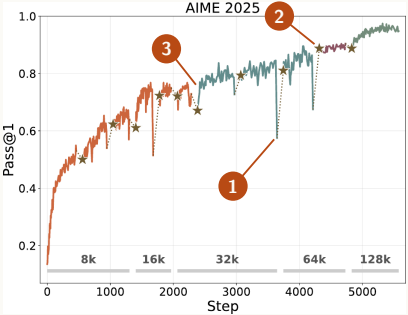



Sampling keeps only the top 97% of the mass. Gradients through the rest made runs diverge in a few steps, so the learner replays the mask.

DeepSeek-V3.2 keeps the sampling mask too.

**Reward.** $R = R_{\text{task}} + 0.5\, R_{\text{lang}} - 0.25\, \rho_q |y|/\ell_{\max}$. Non-English words cost reward, as in DeepSeek-R1: mixed-language thoughts came with divergence spikes. Easy problems (high $\rho_q$) pay more for length (Xiang 2025); the length term is off at the 128k stage.

# After a collapse, re-distill instead of rolling back




STEM climb. ⋆: re-distill; colour: base model; grey: length cap. Fig. 15 (p.35).

1 **Collapse.** Small sampler–learner gaps pile up until the score drops. The fault sits in the weights long before the drop, so an earlier checkpoint is not safe.

2 **Re-distill.** Fine-tune a mid-trained checkpoint on about 1M successful rollouts from the climb, then resume RL. The skills return through the traces; the bad numerics do not.

3 **New base.** The same step moves a climb onto a newer base model.

**What made it work:** traces from many late checkpoints; about 1M of them (more narrows the policy); dropout 0.15.

**In the literature.** DeepSeek-R1 ran one such round (RL traces, SFT on the base, RL again); STaR and ReST-EM loop it. MAI did it ten times in this climb.

# GRPO, not PPO: MAI among RL recipes

**Why no critic?** **1** It is as big as the policy: a second 1T MoE. **2** One reward after up to 128k tokens: token values are hard to learn. **3** 16–128 checked answers per prompt: their mean is a cheap baseline.

| Recipe | Ratio per | Clip | KL | Train–inference gap |
|---|---|---|---|---|
| PPO + critic | token | $\pm 0.2$ | yes | – |
| GRPO (DeepSeek) | token, answer mean | PPO-style | yes | – |
| DAPO (Seed) | token, token mean | 0.2 / 0.28 | no | skip all-pass / all-fail groups |
| GSPO (Qwen3) | whole answer | on the answer ratio | no | MoE without routing replay |
| CISPO (MiniMax) | token | clip weight, keep gradient | no | FP32 LM head |
| **MAI** | token, token mean | 0.4 to $2.5{+}k$, cap 50 | no | bf16, routing + top-p replay |

**Best-practice check** (ScaleRL, Meta 2025). Matches: async with $\leq$8 steps of lag, zero-variance filtering, no KL. Differs: CISPO/GSPO beat token clipping; ScaleRL keeps logits in FP32.

# 2026: PPO for long agents, OPD for merging

| Report | RL algorithm | Merging the specialists |
|---|---|---|
| Kimi K2.5 (Feb) | critic-free, log-ratio masking | not stated |
| GLM-5 (Feb) | GRPO + IcePop, no KL | on-policy distillation (OPD), last |
| DeepSeek-V4 (Apr) | GRPO for each specialist | OPD replaces the mixed RL stage |
| **MAI-Thinking-1** (Jun) | token-level GRPO, wide clip | SFT on teacher traces, short RL |
| GLM-5.2 (Jun) | **critic-based PPO** for long-horizon tasks | parallel OPD of 10+ experts, ∼2 days |

**WHY GLM-5.2 BROUGHT THE CRITIC BACK**

Compaction cuts each run into a different number of sub-traces: groups stop being comparable. A critic scores one rollout.

**WHAT ON-POLICY DISTILLATION CHANGES**

The student writes its own answers; the teacher grades every token. SFT copies the teacher's answers, which the student would not write.

**Algorithm: MAI sits with the 2026 majority. Merging: peers moved to OPD.**

# 2026: drop drifted tokens, don't clip them

**The shared problem** (Part 10): sampler and learner probabilities differ. What happens to a token whose ratio drifted far?

**GLM-5** **Two ratios.** $r = \pi_\theta / \pi_{\text{old}}$ keeps the usual PPO clip; a second ratio $\rho = \pi_{\text{train}} / \pi_{\text{infer}}$ measures the kernel gap, and tokens with $\rho$ outside $[1/\beta, \beta]$ are dropped (IcePop).

**GLM-5 async** **Sampler as $\pi_{\text{old}}$**, as in MAI, but any token outside $[1 - \epsilon_\ell, 1 + \epsilon_h]$ is masked out entirely.

**Kimi K2.5** **Mask by the log-ratio, whatever the advantage's sign**, plus a $\tau(\log r)^2$ penalty that pulls ratios back toward 1.

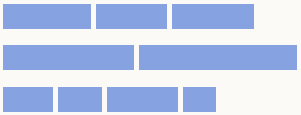
**DeepSeek-V4** **Sample with the serving weights**: rollouts use the same native FP4 weights as deployment.

**MAI** **One ratio, PPO rule**: clip depends on the advantage's sign; wide band 0.4–$2.5{+}k$, cap 50; bf16 and replay shrink the gap.

**The trend.** A far-drifted token is *untrustworthy data*: removed, whatever its advantage. PPO's clip only stops pushing one way. MAI keeps the PPO rule and leans on tight numerics.

# Long agent runs bring the critic back

**Same prompt, three rollouts, after compaction**




different counts and lengths: no fair group, so no group-mean baseline

1. **Compaction.** A run too long for the context is compacted and continued: one run, several pieces.
2. **GLM-5.2's fix.** PPO with a critic: every piece trains on its own, and the critic gives token-level advantages from a single rollout.
3. **The price.** A critic to train next to the policy; critics are hard to get right (2026 work on "value flattening").

### REWARD HACKING, GLM-5.2

Rules flag suspicious tool calls; an LLM judge checks intent. The hack gets dummy output and the run continues; rejecting whole runs destabilized training.

### MANY AGENTS, KIMI K2.5

RL trains an orchestrator to split work over sub-agents; cost is "critical steps" (main agent plus slowest sub-agent), which rewards parallel work.

# OPD: how DeepSeek-V4 and GLM-5 merge

$$\mathcal{L}_{\text{OPD}} = \sum_i w_i \, D_{\text{KL}}\left(\pi_\theta \,\|\, \pi_{E_i}\right)$$

The student writes its own answers; teachers score every token; the student moves toward them (reverse KL, DeepSeek-V4 Eq. 29).

### GLM-5: REUSE THE RL LOSS

Per-token advantage $= \log \pi_{\text{teacher}} - \log \pi_\theta$ (no gradient through it) in the RL loss. Group size 1: the teacher is the baseline. Used last, to win back skills lost across RL stages.

### DEEPSEEK-V4: FULL VOCABULARY

The per-token estimate was noisy and unstable, so V4 matches the teachers' whole next-token distributions. 10+ teachers, loaded on demand. Replaces the mixed RL stage.

**vs. MAI.** MAI never discusses OPD. It merges by SFT plus short RL (Part 9), keeping STEM in that RL because reasoning otherwise decays: the loss GLM-5 undoes with OPD.

PART 9 OF 10

# Three climbs

**③ Keep RL alive**

Three skills, three reward designs, one final model.

**IN THIS PART**

The pipeline · STEM keys · SWE environments · safety gate

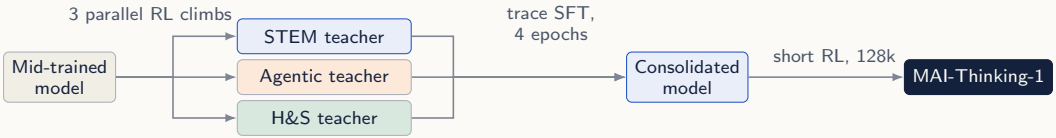

**PRE-TRAINING**

1 Model architecture
2 Ablations: what transfers
3 From ladder to real run
4 Pre-training data
5 Mid-training
6 Training recipe
7 YOLO training infrastructure

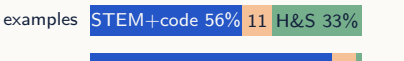
**POST-TRAINING**

8 RL key technologies
**9 Three climbs**
10 RL infrastructure: Rocket

# Three specialists climb in parallel, then merge




**Consolidation SFT mix**




11, 9: agentic. H&S: helpfulness & safety.

**Q Why mix by examples, not tokens?**

STEM traces are over 20× longer than chat, so H&S is 2% of tokens, yet it held up.

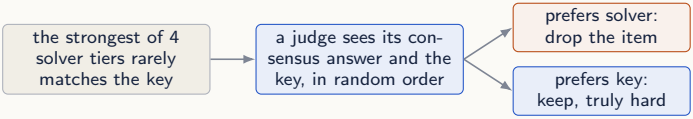
**Q Why a last RL stage?**

To polish safety, over-refusal and style. Some STEM stays in, or hard reasoning slowly decays.

**In the literature.** GLM-4.5 also trains expert models and fuses them by self-distillation; DeepSeek-V3.2 distills specialists, then runs one mixed RL stage.

# A STEM reward is only as good as its answer key

## BLIND CHECK OF A HARD ITEM'S KEY




**Reward:** the final answer, checked by SymPy, an AI judge, or unit tests.

### THE QUESTION

Textbooks, forums and contests hold millions of question–answer pairs. Can their keys be rewards?

### WHAT THEY DID

Drop the uncheckable and benchmark repeats; rewrite multiple choice and proofs as open questions.

### WHAT HAPPENED

Over 5M items, 550k of them hard, plus 160k coding tasks with tests. AIME 2025: 0.14 to 0.96.

### SO WHAT

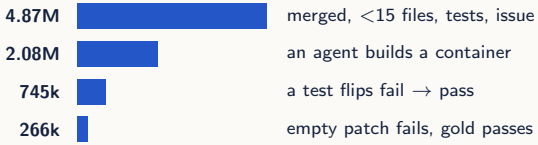
A guessable format pays for guessing; a wrong key punishes the right answer.

**In the literature.** Big-Math (Albalak 2025) also turns multiple choice into open questions. DeepSeek-R1 and Tülu 3 made rule-checked rewards standard; MAI also checks the key.

# Agentic RL is mostly environment engineering

A rollout is many turns of bash and file edits in a fresh container; hidden tests then grade the patch.

From 102M public GitHub PRs, keep those:




5.5% survive, from 94k repos. The builder yields about 20 verified environments a minute (App. G).

**Hacks the monitor caught, and the fix**

| Hack | Fix |
|---|---|
| Find the merged PR online | No network, or a minimal allowlist |
| Read the fix in later git commits | Scrub every commit after the base |
| Edit or patch the tests | Reset test files; add hidden tests only at grading |

**Mixing in STEM** steadied the climb and helped SWE. SWE-bench Verified: about 34% → 74%. **General tool use** adds 150+ synthetic environments with 130k tasks.

**In the literature.** Harvesting follows SWE-rebench; built but rejected environments get synthetic bugs, as in SWE-smith and BugPilot. SWE-bench itself once leaked fixes through git history.

# Safety is a gate, not a weight

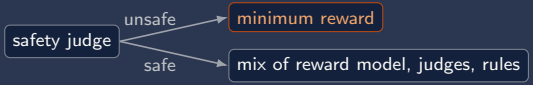
**87.8%**

of answers that broke policy still got 3 or more (of 5) from the reward model.

So in a weighted sum, a polished unsafe answer can win.

Style judges grade 0/1/2 and act only after safety and rules pass: coarse grades were harder to hack.

**Gate.** Safety is judged first; an unsafe answer is never graded on quality.




**Priority.** In instruction following, rule checks and rubric judges come first; the reward model only splits ties. GRPO compares answers within a group, so reward scales never need matching.

**In the literature.** Llama 2 switched to its safety reward model below a threshold; Safe RLHF (Dai 2023) trades safety against help with a Lagrange multiplier.

PART 10 OF 10

# RL infrastructure: Rocket

② ③ **Machine + RL**

Generation dominates RL cost: keep 4,096 GPUs busy and the learner consistent with them.

**IN THIS PART**

Async actors · replay · weight sync · vs. open RL stacks


**PRE-TRAINING**

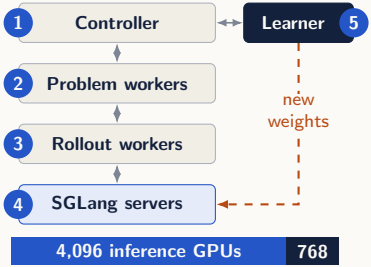
1 Model architecture
2 Ablations: what transfers
3 From ladder to real run
4 Pre-training data
5 Mid-training
6 Training recipe
7 YOLO training infrastructure

**POST-TRAINING**

8 RL key technologies
9 Three climbs
10 **RL infrastructure: Rocket**

# Async actors keep 4,096 inference GPUs busy

**Why async.** A synchronous step waits for its slowest rollout (up to 128k tokens), and inference idles during training. Rocket splits the loop into Ray actors linked by queues, so no stage blocks on the slowest rollout.

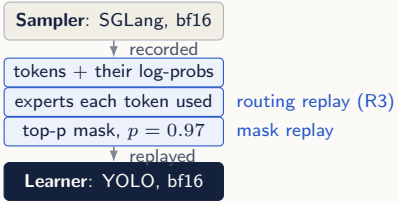



largest job: 4,864 GB300, about 5 : 1

1. **Controller** deals out tasks and batches finished groups.
2. **Problem workers**, a pool: 16 rollouts, then 128 if the pass rate is in range; score each against its group (advantage).
3. **Rollout worker**: one rollout, model and tools in a loop, graded.
4. **SGLang router** balances load and caches shared prompt prefixes.
5. **Learner** pushes new weights every 5 steps; rollouts more than 8 pushes old are dropped.

**Compare.** MAI says TRL, OpenRLHF and verl (HybridFlow 2024) could not run async RL on thousands of GPUs; verl, slime, AReaL and SkyRL now all offer disaggregated async modes.

# The learner replays the sampler's choices




**What it bought.** Bf16 on both sides beat lower precision, and replay made collapses (Part 8) rarer.

**Q What goes wrong?**

Different kernels make the *importance ratio* $\pi_\theta/\pi_{\text{old}}$ drift; long rollouts add it up. Eq. 5 takes $\pi_{\text{old}}$ from the sampler, so one ratio carries both staleness and kernel mismatch.

**Q Why replay the expert choices?**

A slightly-off probability skews the ratio a little; an expert choice that flips in a near-tie skews it a lot.

**Q Why mask the tail?**

The sampler renormalizes over the top 97%. The learner's full softmax skews every ratio and pushes gradient into logits the sampler never picks; runs diverged in a few steps.

**Compare.** verl, slime and SkyRL instead recompute $\pi_{\text{old}}$ on the learner and correct the sampler gap with a separate truncated IS weight (Yao 2025). DeepSeek-V3.2 also replays routing and the mask.

# Plan the weight sync once, reuse it every push

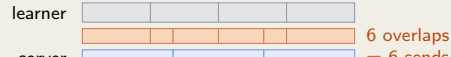
**The problem.** Every 5 steps, new weights must reach thousands of waiting GPUs that shard tensors differently.

### OPENRLHF, SLIME (NCCL MODE): GATHER, THEN BROADCAST

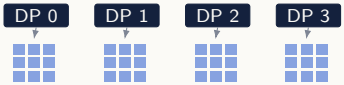



Assemble each tensor, send it whole to every engine. Simple, but every byte goes everywhere.

### ROCKET: A COMPILED TRANSFER PLAN




At job start, intersect both layouts: one entry per overlap (from, to, bytes, cast). Send only those.




**Scaling out.** Planned for one learner–server pair, then expanded to live replicas that can join or leave. Each data-parallel (DP) group holds a full copy: four groups feed 36 servers at once; a failure stays local.

**Compare.** Kimi K2 broadcasts full weights. Perplexity's RDMA transfer (2025) is closest: point-to-point writes on a precomputed schedule; SGLang added a peer-to-peer RDMA weight-update path in 2026.

# Rocket vs. open RL stacks: match, don't correct

| | Open stacks: all now run async RL on split pools | Rocket |
|---|---|---|
| **GPUs** | verl shares one pool by default; slime, AReaL, SkyRL also split. | Split only: 4,096 inference, 768 learner. |
| **Staleness** | Capped up front: generation waits if too far ahead (AReaL: 2–8 versions). | Generate freely; drop what is over 8 versions old. |
| **$\pi_{\text{old}}$ from** | Recomputed by the learner; the sampler gap gets its own weight (truncated IS, AReaL's decoupled loss). | The sampler's own probabilities; the learner is made to match (bf16, routing and top-p replay). |
| **Weight sync** | Gather, then NCCL broadcast; CUDA IPC when colocated. | Compiled point-to-point plan. |

**Rocket makes the learner reproduce the sampler, not correct for it.**

**Gain:** no extra forward pass of a 1T model over rollouts up to 128k tokens. **Price:** one ratio holds staleness and kernel error together, so numerics must stay tight: the replay of two slides back.

CLOSING · OPEN QUESTIONS

# What the paper measures vs. only asserts

> **Pre-training choices get efficiency-gain tables; most RL choices get "we found".**

**Q How much did each RL choice matter?**
Band, entropy knob, cap and replay are argued, not ablated.

**Q Is the base model's lead home advantage?**
Its loss sets are the suite the mix search minimized.

**Q How much of the agentic gap is harness?**
Sonnet's agentic scores come from Anthropic's own harness.

**Q What did ten re-distillations cost?**
SFT compute and RL steps spent recovering are not reported.

**Q Was the mix checked across enough scale?**
Finalists were re-trained at $2.8\times$ compute; theory says the best mix keeps moving (Part 4).

**Q What was never tried?**
Muon and $\mu$P-style transfer; the ladder varies depth only.

CLOSING · THE THREE CLAIMS

# What to take away

1. **Decide small, check big.** Price each change in compute saved, at several sizes, on held-out loss weighted toward what you care about. Keep it only if it holds in wall-clock time and after RL: MAI's own flips (data rankings, FLOP wins that lose on hours) show why.
2. **Design for the machine.** Pick the layout by step time, not FLOPs. Make every restart checkable bit for bit, and plan the weight sync once.
3. **Keep RL alive.** One wide clip and an entropy knob instead of a KL term and a bonus; a learner that reproduces the sampler; re-distill after a collapse; merge the specialists by distillation; safety as a gate.

**Where it lands.** At 35B active parameters and without distilling other labs' models: ahead of Sonnet 4.6 on AIME 2025, behind on GPQA and agentic coding. **What I would copy first:** the efficiency-gain yardstick and bit-exact restarts. Both are cheap and pay off at any scale.

BACKUP

# Backup slides

1 · DESIGN LINEAGE

# Where each piece comes from

| | Gemma 3 (2025) | LatentMoE (NVIDIA, 2025) | DeepSeekMoE (2024) |
|---|---|---|---|
| **Idea** | 5 sliding-window layers per global one | Send tokens and run experts at a smaller, latent width | Many small experts, plus a **shared expert** every token uses |
| **In MAI** | Same 5:1, window 512 (Gemma: 1024); global layers use **no position encoding** | 6656 → 3072, sized for DeepEP's all-to-all; the **router still reads all 6656** | 512 small experts kept; **shared expert dropped** |
| **Why** | Only 13 of 78 layers cache all past keys and values: ≈53 KB per token in BF16, 6× less than all-global | Fewer bytes sent, smaller experts: what made 512 experts affordable | Only helps an all-MoE stack (per FLOP: 6% behind MAI's layout → 3% ahead) |

**WHY DROP THE SHARED EXPERT?**

An always-on dense path; MAI's dense FFN layers already are one, so it added little or nothing.

**Compare.** Qwen3's MoE models also dropped the shared expert. Llama 4 Maverick alternates dense and MoE layers like MAI, but keeps one.

# Same four GPUs, four kinds of parallelism

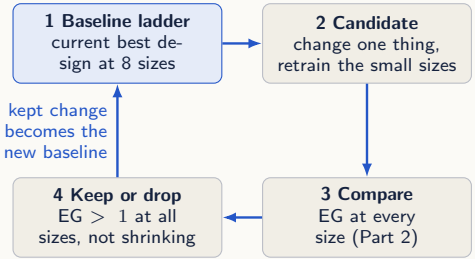
| | 4 GPUs | Each GPU holds | What moves | Collective |
|---|---|---|---|---|
| **DP** | $W$ $W$ $W$ $W$ | the **whole model**; different data | **gradients**, the same shape on every GPU | **all-reduce**: sum, everyone gets the total |
| **ZeRO** | $\frac{1}{4}$ $\frac{1}{4}$ $\frac{1}{4}$ $\frac{1}{4}$ | **1/4 of params and optimizer**; different data | gradients, then parameters | **reduce-scatter** grads (sum, keep 1/4); **all-gather** params |
| **TP** | $W_0$ $W_1$ $W_2$ $W_3$ | **a slice of every matrix**; the same tokens | **partial outputs** of each layer | **all-reduce** every layer, on the critical path |
| **EP** | $E_{01}$ $E_{23}$ $E_{45}$ $E_{67}$ | **different experts**; different tokens | **the tokens themselves**, a different subset to each GPU | **all-to-all**: no sum, a permutation; twice per MoE layer |

**All-reduce sums copies; all-to-all reshuffles different tokens, twice per MoE layer.**

**In MAI.** EP of 64 inside one NVLink rack keeps the all-to-all on fast links; ZeRO runs across racks over InfiniBand. Origins: ZeRO (2020), Megatron-LM TP (2019), GShard EP (2020).

2 · HOW THE LADDER IS USED

# The ladder is a loop: test, compare, keep, repeat




## VERDICTS THIS LOOP PRODUCED (PART 2)

**KEPT: MORE EXPERTS**

EG $> 1$ and growing with compute.

**DROPPED: MOE IN EVERY LAYER**

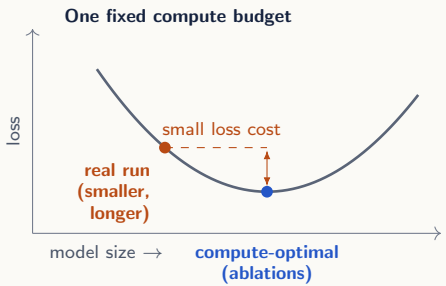
Ties per FLOP, loses per training hour (0.82).

**DROPPED: THE STEM-HEAVY MIX**

Wins at 5B, loses at 23B.

**In the literature.** AI2's OLMo uses a ladder the same way. DataDecide (ICML 2025): 150M rankings pick the 1B winner ∼80% of the time.

# Ablate near compute-optimal, train far past it




Same FLOPs at every point. Schematic.

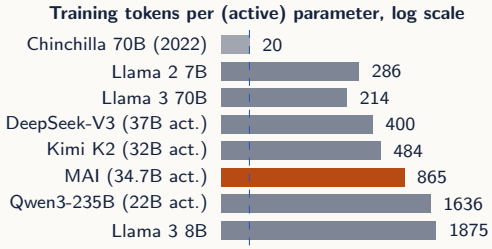
**Q What is TPP?**
Training tokens ÷ active parameters. MAI: 30T / 34.7B ≈ 865.

**Q What is compute-optimal?**
For a fixed FLOP budget, the size/token split with the lowest loss (Chinchilla): 100–200 TPP here. Ablations run there.

**Q Why train the real model far past it?**
Serving cost grows with model size. A smaller model trained longer (500–1,000 TPP) costs a little loss per FLOP, saves on every token served (Sardana 2024).

**Same shape, different length: ablations must also copy how often each source repeats.**

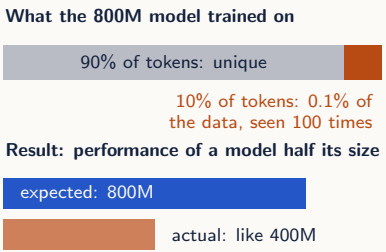
# Why real runs train far past Chinchilla's 20




Dashed: Chinchilla's 20. Tokens from each report (Qwen3: 36T; Llama 3: 15T).

1. **Chinchilla minimizes training compute only.** Serving cost grows with model size, for years; RL rollouts are serving too. Sardana (2024): train smaller and longer.
2. **The valley is flat.** Leaving the optimum costs little, and loss keeps falling: Llama 3 8B's optimum is ∼200B tokens, yet it improved to 15T.
3. **MoE counts active parameters.** More experts shift the optimum to more tokens (Ludziejewski, ICML 2025); MAI's sits at 100–200.
4. **The limit is unique data.** Past it, more tokens means repeats (next slides).

# A small slice repeated often is the danger




**Q The question**
Corpora contain duplicated slices, and labs up-weight good data. How much can that hurt?

**Q The finding**
0.1% of the data seen 100 times made an 800M model perform like a 400M one, with 90% of tokens still unique.

**Q Why**
Memorizing the slice uses up capacity meant for everything else; the damage peaks at a middle range of repetition.

**Q For MAI**
The near-duplicate STEM sources behind Part 2's flip fit this case: hence hard dedup and per-source caps.

# Mixing and regularizing make repeats safer

### APPLE (SEDOVA, SETO, ABLIN, 2026)

**Question.** A scarce domain is mixed into plenty of generic data. How often can it repeat?

**Finding.** 15–20 times before overfitting, versus $\sim$4 alone: the fresh generic data regularizes. Bigger models overfit sooner.

**For MAI.** Math read 5.3 times inside a mixture is comfortably safe.

### STANFORD (LIANG, HASHIMOTO, ICLR 2026)

**Question.** Data fixed, compute unlimited: how to train?

**Finding.** More epochs and bigger models overfit unless regularized hard: best weight decay $\approx$30$\times$ the usual. With ensembles, the same loss from 5.17$\times$ less data.

**For MAI.** Dropout 0.15 on top of weight decay is the same instinct.

**Fine in moderation, dangerous in a small slice, safer in a mixture with strong regularization.**

# A mixing law turns the search into curve fitting

### Loss is a smooth function of the mix

Ye et al., Fudan, ICLR 2025

$$L_i(r) = c_i + k_i \exp\left(\sum_j t_{ij}\, r_j\right)$$

1. $r_j$: share of source $j$; $c_i$: loss on domain $i$ that no mix removes.
2. $t_{ij} < 0$: training on $j$ helps $i$; $t_{ij} > 0$: $j$ crowds $i$ out. MAI's Fig. 7 shows such signs: PDFs and web math lower physics loss; code does not.
3. Fit it on a few dozen mixes, then minimize it.

### Where model scale enters

**RegMix** Fit at 1M parameters and assume the ranking of mixes holds at any size. Checked only up to 1B (ICLR 2025).

**Nested** Ye et al.: first push each mix's loss to the target size and length with scaling laws, then fit the mixing law there.

**MAI** RegMix-style: thousands of runs at 0.76–4B active, then a few finalists at $2.8\times$ the compute.

> **Aioli (2025): every method minimizes loss under an assumed law. With its parameters off, none reliably beats plain stratified sampling.**

# Put the data in mid-training; let RL refine

### PRISM (IBM, 2026): 7 BASES, 3–24B

**Question.** Should domain data go into mid-training or into RL?

**Finding.** ∼27B mid-training tokens, then RL: average 3–4× higher; RL alone left AIME near zero. Science data in mid-training: +17–28 GPQA after RL; changing the RL mix: under 2.

**Why.** Mid-training rewrites over 90% of the weights; RL touches about 5%.

### CMU (ZHANG, NEUBIG, YUE, 2025)

**Question.** In a fully controlled setup, what does each stage really add?

**Finding.** RL creates new ability only at the model's edge of competence: tasks that are hard but still reachable. At a fixed compute budget, mid-training plus RL beats RL alone.

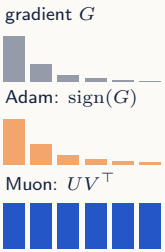
**For MAI.** The 10–80% pass-rate filter (Part 8) keeps RL at that edge.

**Mid-training sets the starting point and the skills on offer; RL does the climbing.**

# Why a layer wants the spectral ruler

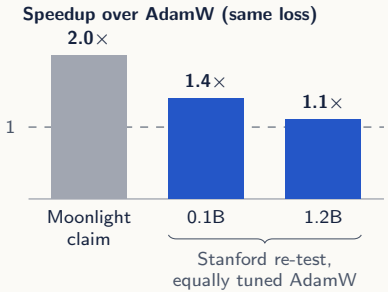
1. **A layer is a map.** $\|\Delta W x\| \leq \|\Delta W\|_{\text{op}} \|x\|$: capping the spectral norm caps how far any feature moves, at any width (Yang, Simon, Bernstein 2023; basis of $\mu$P).
2. **Gradients are nearly low-rank.** A few directions dominate, and the sign step keeps that shape (sign of a rank-1 matrix is rank-1).
3. **Muon flattens the spectrum.** Rare directions get the same step. Moonlight: Muon's weights have higher SVD entropy in over 90% of matrices.
4. **The ruler explains the costs.** Needs the whole matrix (sharding); fits only 2-D maps (embeddings, head stay on AdamW); $QK$ logits can grow (QK-Norm or QK-Clip).

**Singular values of the step (schematic)**




**Also.** Sign descent is *also* steepest descent in the $\ell_1 \to \ell_\infty$ operator norm (Bernstein & Newhouse): a matrix norm, but it measures activations by their largest entry, not their RMS.

# Does Muon pay off? Less as models grow




**for** **Moonlight (Moonshot, 2025):** ∼2× compute efficiency at compute-optimal; scaling it needed weight decay and a matched update size.

**for** **Essential AI (2025):** keeps data efficiency at much larger batches, so it wins on wall-clock time.

**against** **Stanford (Wen, Hall, Ma, Liang, 2025):** with AdamW tuned as carefully, matrix optimizers give 1.4× at 0.1B and only 1.1× at 1.2B.

**caution** **Moonlight again:** switching optimizer between pre-training and fine-tuning hurts, in both directions.

**Real gains that shrink with scale, plus engineering costs (next slide).**

# Who switched to Muon, and what they had to fix

### KIMI K2 (MOONSHOT, JUL 2025): MUONCLIP

Plain Muon at 9B/53B pushed attention logits past 1000. **Fix:** QK-Clip rescales $W_q, W_k$ when a head's max logit passes a threshold.

### GLM-4.5 (ZHIPU, AUG 2025)

Muon for all but embeddings, biases and norms; update RMS 0.2. Faster convergence, larger batches tolerated.

### DEEPSEEK-V4 (APR 2026): ADAMW → MUON

V3 used AdamW. **Fix:** RMSNorm on queries and KV bounds logits (no QK-Clip); ZeRO buckets keep each matrix whole.

### QWEN3.8-NEXT (2026)

Muon only on true 2-D maps. **Fix:** router stays on AdamW (Muon destabilized it); fused qkv and fc1 split before orthogonalizing.

**Common pattern.** AdamW kept for embeddings and head · a logit guard (QK-Norm or QK-Clip) · ZeRO rebuilt for whole matrices.

# Why AdamW was still the safer bet for MAI

> **MAI never discusses Muon. Read against the reports above, its stack makes AdamW the cheaper and safer bet, not the faster optimizer.**

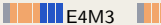
1. **Calibration.** The ladder, the EG yardstick and every tuned knob were built on AdamW runs; switching means redoing them.
2. **Sharding.** YOLO shards optimizer state with ZeRO; Muon needs whole matrices, which DeepSeek-V4 and Qwen had to rebuild ZeRO for.
3. **The yardstick is post-RL.** Muon's evidence is pre-training loss, and its gain shrinks with scale (Stanford, 2025).
4. **The RL learner runs AdamW** ($\beta_1 = \beta_2 = 0.95$). Moonlight saw that switching optimizers between stages hurts.

**Worth knowing.** Microsoft Research's own Dion (Ahn et al., 2025) replaces Newton–Schulz with power iteration that works on sharded weights: the exact conflict in point 2.

# FP8 for matmuls, FP32 where errors add up

**The problem.** FP8 matmuls run at twice the BF16 speed, but an 8-bit number keeps only 2–3 bits of precision.

### FP8: THE BIG MATMULS




Forward: E4M3. Activation gradients: E5M2 (more range). Scale factors track the max values of the last 1024 steps.

### BF16: THE DEFAULT

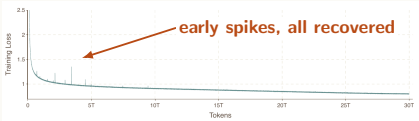



Weights, activations, weight-gradient matmuls. Random (stochastic) rounding keeps tiny gradients.

### FP32: WHATEVER ADDS UP



Residual stream, attention scores, all logits, AdamW and its state, gradient sums.



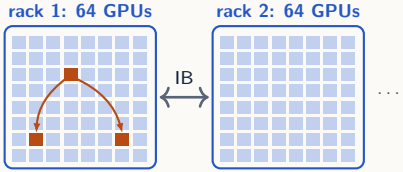

Pre-training loss, 30T tokens, unsmoothed (Fig. 9).
Bits: sign exponent mantissa.

**What happened.** A few early spikes, each recovered on its own; no skipped batches, no manual changes. The spikes hit code data during bursts of expert imbalance.

**Compare.** DeepSeek-V3: E4M3 everywhere, per-block scales computed on the fly. MAI saves its 1024-step scale history in every checkpoint, as bit-exact restarts need.

# YOLO: one in-house stack, pre-training to RL

**Why build one?** Five model generations each changed kernels, sharding and scheduling, and runs had to be bit-exact.




■ **Inside a rack** (64 of 72 GPUs in use): 512 experts, 8 per GPU; the all-to-all (tokens to their experts and back) stays on NVLink.

↔ **Across racks:** copies, synced over InfiniBand (IB).

YOLO, written on plain PyTorch, runs pre-training, SFT and the RL learner. It borrows ideas, then changes them:

**Megatron-Core** took tensor parallel, parallel folding
Parallelism per tensor: tensor-parallel for embeddings, loss, attention; none for MLP and experts.

**DeepSpeed** took ZeRO 1–3, Ulysses long-context split
ZeRO (weights, optimizer state split over GPUs) rebuilt: always sharded, contiguous buffers, fewer collectives.

**DTensor, JAX, DCP** took sharding tags, checkpointing
Tags only describe a layout, never insert communication. PyTorch DCP rewritten: saves over 10× faster.

**Built new**
FP8 and grouped-expert kernels, an NVLink all-to-all, dropless MoE with router replay for RL.